

# Understanding KV Cache

From Autoregressive Decoding to Modern LLM Inference

---

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 02 - Understanding AI Series

# What this handbook will make clear

---

A KV cache is easy to describe badly. It is often called “LLM memory,” a “speed trick,” or simply “cached attention.” Those shortcuts hide the mechanism that matters.

This handbook builds the idea from first principles. By the end, you should be able to explain:

- why past keys and values are reusable in an ordinary causal Transformer;
- exactly what tensors are stored and how their shapes grow;
- why cache memory scales linearly with cached tokens, layers, KV heads, head dimension, and bytes per element;
- why MQA, GQA, and MLA attack KV cost in different ways;
- why cache management becomes a systems problem under concurrent serving;
- what PagedAttention, prefix caching, offloading, and KV-cache quantization actually change.

## Prerequisites

You should already know the basic attention equation and the role of Q, K, and V. The previous handbook, *Understanding Attention*, is the natural prerequisite.

*The goal is not to memorize another optimization. It is to understand why the optimization is mathematically valid, what resource it saves, and what costs remain.*

# Why decoding is sequential

---

A decoder-only language model assigns a distribution to the next token conditioned on the tokens already available:

$$p(x_{t+1} \mid x_{1:t}).$$

After a token is selected, it is appended to the sequence and becomes part of the context for the next step. Generation therefore forms a dependency chain:

$$x_1, \dots, x_t \rightarrow x_{t+1} \rightarrow x_{t+2} \rightarrow \dots$$

## Two execution phases

For a prompt of many tokens, the model can process those prompt positions together under a causal mask. This is usually called **prefill**. Once generation begins, the model usually advances one newly generated token per sequence per decode step. This is the **decode** phase.



This sequential decode structure is the setting in which KV caching becomes crucial.

## What a naive decoder would repeat

---

Consider one self-attention layer. For every token representation, the layer forms learned projections:

$$q_i = h_i W_Q, \quad k_i = h_i W_K, \quad v_i = h_i W_V.$$

At decode step  $t$ , the new query must attend to the keys and values of positions  $1, \dots, t$ .

A naive implementation could rebuild the old  $k_i$  and  $v_i$  tensors from scratch on every step. That would repeatedly perform projections whose inputs have not changed.

### The useful observation

For an ordinary causal Transformer, once position  $i$  has been processed at a fixed layer, adding future tokens does not retroactively change that position's layer state. Causal masking prevents information from future positions from flowing backward into the representation of an earlier token.

Therefore the key and value derived for an earlier position can be retained and reused on later decode steps.

*KV caching is valid because past layer states are invariant to future tokens under the model's causal computation. It is not a heuristic guess.*

## What exactly is cached?

---

Suppose the model has already processed positions  $1, \dots, t-1$ . At one attention layer, it has stored

$$K_{1:t-1} = [k_1; \dots; k_{t-1}], \quad V_{1:t-1} = [v_1; \dots; v_{t-1}].$$

For the new position  $t$ , the layer computes only the new projections

$$q_t, \quad k_t, \quad v_t,$$

then appends the new key and value:

$$K_{1:t} = [K_{1:t-1}; k_t], \quad V_{1:t} = [V_{1:t-1}; v_t].$$

The current query attends over the accumulated history:

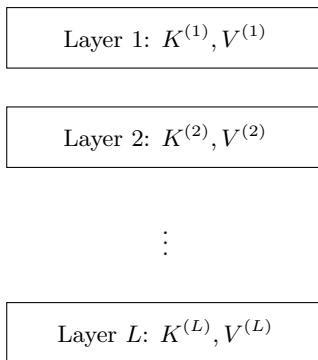
$$\text{softmax}\left(\frac{q_t K_{1:t}^T}{\sqrt{d_h}}\right) V_{1:t}.$$

### What is not cached by this idea

The cache does not store “thoughts” or symbolic facts. It stores numerical key/value states for each attention layer and cached token position, in whatever physical layout the inference engine uses.

# The cache is not a single matrix

Each self-attention layer has its own learned projections and therefore its own K/V state. A model with  $L$  attention layers maintains a logical history like:



For a common logical layout, one layer's cache can be thought of as

$$K, V \in \mathbb{R}^{B \times H_{KV} \times T \times D_h},$$

where  $B$  is batch size,  $H_{KV}$  is the number of key/value heads,  $T$  is cached sequence length, and  $D_h$  is head dimension.

Actual kernels may store or pack dimensions differently. The logical shape is the important mental model; a particular physical layout is an implementation detail.

# The cache is created differently in the two phases

## Prefill

Given an input prompt, the model computes K/V states for many prompt positions in parallel at each layer, subject to causal masking. These states populate the initial cache.

## Decode

For each newly generated token, each layer computes a new  $q_t, k_t, v_t$ , appends  $k_t, v_t$ , and performs attention for the new query against the cached history.

|                                | Prefill                   | Decode                                  |
|--------------------------------|---------------------------|-----------------------------------------|
| <b>New positions processed</b> | Many prompt tokens        | Usually one new token per sequence/step |
| <b>Cache action</b>            | Build initial K/V history | Append new K/V state                    |
| <b>Attention query length</b>  | Many                      | Usually 1                               |
| <b>Reuse benefit</b>           | Establishes state         | Avoids recomputing past K/V projections |

*A KV cache does not eliminate attention over the history. The new query still has to interact with past K/V state for dense attention.*

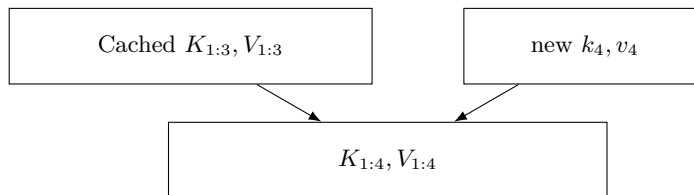
## A four-token example

---

Imagine the cache currently holds positions 1–3. At the next step:

1. The model receives the representation for token 4 at this layer.
2. It computes  $q_4$ ,  $k_4$ , and  $v_4$ .
3. It appends  $k_4$  and  $v_4$  to the layer cache.
4.  $q_4$  is compared with  $K_{1:4}$ .
5. The resulting attention weights mix  $V_{1:4}$ .
6. The layer produces the representation used by the rest of the model.

The old  $k_1, k_2, k_3$  and  $v_1, v_2, v_3$  are read, not regenerated.





# How large is the KV cache?

For a conventional dense attention stack with uniform KV-head count and head dimension, an idealized per-sequence cache size is

$$M_{KV} = 2 L T H_{KV} D_h b$$

where:

- 2 accounts for both K and V;
- $L$  is the number of attention layers;
- $T$  is cached tokens;
- $H_{KV}$  is the number of key/value heads;
- $D_h$  is head dimension;
- $b$  is bytes per stored element.

For  $B$  equal-length sequences, multiply by  $B$ . More generally, with variable cached lengths  $T_i$ , the idealized payload is

$$M_{KV} = 2 L H_{KV} D_h b \sum_{i=1}^B T_i.$$

Real serving engines allocate in blocks and may add padding, scales, metadata, sharding, or model-specific states, so these formulas describe the tensor payload rather than every byte consumed by a runtime.

*The formula is useful because each architectural choice has a visible lever: context length changes  $T$ ; MQA/GQA change  $H_{KV}$ ; cache quantization changes the storage term  $b$  (plus quantization metadata).*

# MHA, GQA, and MQA in one model

Take an illustrative decoder with

$$L = 32, \quad D_h = 128, \quad b = 2 \text{ bytes}, \quad H_Q = 32.$$

The only value we change is  $H_{KV}$ .

| Architecture       | $H_{KV}$ | KiB / token | GiB at 8,192 tokens |
|--------------------|----------|-------------|---------------------|
| MHA                | 32       | 512         | 4.000               |
| GQA (illustrative) | 8        | 128         | 1.000               |
| MQA                | 1        | 16          | 0.125               |

For the GQA row:

$$2 \cdot 32 \cdot 8192 \cdot 8 \cdot 128 \cdot 2 = 1,073,741,824 \text{ bytes} = 1 \text{ GiB}.$$

These are **per-sequence tensor-payload examples**, not universal sizes for all 32-layer models. Different models use different layer counts, KV heads, dimensions, dtypes, local-attention patterns, and sharding.

## Why long context changes serving economics

---

Holding everything else fixed,

$$M_{KV} \propto T.$$

That means doubling the number of cached tokens doubles the idealized KV tensor payload.

Using the same illustrative model as the previous page:

| Architecture          | 8K        | 32K     | 128K   |
|-----------------------|-----------|---------|--------|
| MHA ( $H_{KV} = 32$ ) | 4 GiB     | 16 GiB  | 64 GiB |
| GQA ( $H_{KV} = 8$ )  | 1 GiB     | 4 GiB   | 16 GiB |
| MQA ( $H_{KV} = 1$ )  | 0.125 GiB | 0.5 GiB | 2 GiB  |

A model's parameter memory does not grow just because one request has more context, but its per-request KV state does. This is why long-context capability creates a serving-capacity problem even when the model weights are unchanged.

*A context-window specification tells you what the model can accept. It does not tell you how many long-context requests a particular server can hold concurrently.*

## And what it does not save

---

KV caching avoids recomputing the historical key/value projections and the earlier layer states needed to produce them. That is a major reduction in redundant work during autoregressive decoding.

But for ordinary dense attention, the new query still needs access to past K/V state. As  $T$  grows, the attention operation for the new position still interacts with a longer history.

### Three different resources

Do not collapse these into one claim:

- **Computation:** arithmetic required by projections and attention.
- **Capacity:** bytes needed to keep K/V state resident.
- **Memory traffic/bandwidth:** bytes moved to feed the attention kernel during decoding.

MQA was explicitly motivated by the memory-bandwidth cost of repeatedly loading large K/V tensors during incremental decoding, not merely by “using fewer parameters.”

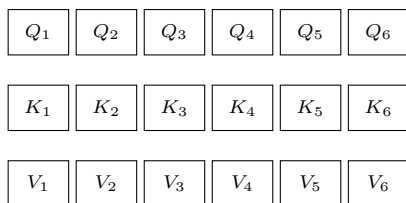
## The expensive baseline for KV state

In standard multi-head attention, each query head has a corresponding key head and value head. Conceptually:

$$H_{KV} = H_Q.$$

If there are 32 query heads, there are 32 key heads and 32 value heads.

During decode, each new query head attends to the historical K/V state associated with its head. This gives the architecture rich per-head K/V projections, but it also makes the KV cache grow in direct proportion to the number of KV heads.



This page is the baseline against which MQA and GQA make sense.

## Share K/V across query heads

---

Multi-query attention (MQA) keeps multiple query heads but shares a single key head and a single value head across them:

$$H_Q > 1, \quad H_{KV} = 1.$$

Noam Shazeer's 2019 paper motivates this change specifically around incremental decoding: loading K/V tensors can become a memory-bandwidth bottleneck, so shrinking those tensors can improve decoding efficiency.

### What changes

The query heads remain distinct. The reduction comes from K/V sharing, not from deleting query heads.

### What the cache formula says

If an MHA model with 32 query/KV heads moved to one KV head while keeping the same  $L, D_h, T, b$ , the KV tensor payload would be reduced by a factor of 32.

That arithmetic is an architectural consequence. Whether a particular MQA model preserves quality relative to a particular MHA baseline is an empirical question, not a mathematical guarantee.

## A middle point between MHA and MQA

Grouped-query attention (GQA) uses fewer KV heads than query heads, but more than one:

$$1 < H_{KV} < H_Q.$$

Query heads are partitioned into groups, and the query heads in a group share the same K/V head.

|             | MHA                 | GQA           | MQA                |
|-------------|---------------------|---------------|--------------------|
| Query heads | many                | many          | many               |
| KV heads    | same as Q           | intermediate  | 1                  |
| K/V sharing | none across Q heads | within groups | across all Q heads |

Ainslie et al. introduced GQA as a generalization of MQA and reported that their uptrained GQA models achieved quality close to MHA with speed comparable to MQA in their experiments.

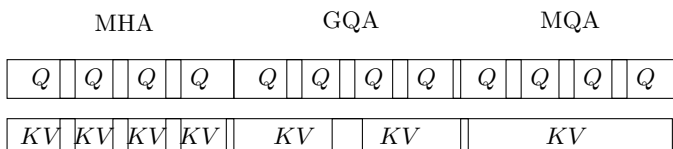
*“GQA is as good as MHA” is too broad. The defensible statement is about the reported models and experimental setup in the GQA paper.*

# Why KV-head count matters so much

For fixed  $L, T, D_h, b$ ,

$$M_{KV} \propto H_{KV}.$$

So the head-sharing choice is visible directly in memory consumption.



This is why KV cache cannot be discussed independently of attention architecture. A model with the same parameter scale and context length can have very different cache requirements depending on its K/V representation.



# Compress the representation instead of only sharing heads

---

DeepSeek-V2 introduced Multi-head Latent Attention (MLA) as a different route to reducing KV-cache cost. Its central idea is to represent the non-positional K/V information through a lower-dimensional latent state. At inference, projection absorption lets the model avoid materializing and caching the full per-head K/V tensors in the conventional way.

This is conceptually different from MQA/GQA:

- **MQA:** one shared K/V head;
- **GQA:** a smaller number of shared K/V heads;
- **MLA:** low-rank latent compression of K/V information.

DeepSeek-V2 reports a 93.3% KV-cache reduction relative to its stated comparison architecture. That number is a paper-specific result, not a universal “MLA compression ratio.”

## RoPE complication

DeepSeek-V2 also uses decoupled rotary positional components because ordinary RoPE interacts with the projection-absorption strategy used by MLA. This is why a simple “MLA caches one tiny vector” summary is incomplete.

## What happens to RoPE and cached keys?

---

With standard rotary position embeddings, position-dependent rotations are applied to queries and keys before their dot products are used by attention. During ordinary cached decoding, the historical keys stored for future reuse must remain consistent with the positional treatment expected by later queries.

A useful implementation-level mental model is:

- compute the new key for the new position;
- apply the model's positional transformation as required by the attention implementation;
- store/reuse the key representation in the form expected by future attention calls.

Different architectures can change exactly what is stored. MLA is the clearest example: its decoupled positional component changes the cache structure compared with a conventional MHA/GQA model.

*Do not hard-code “the KV cache always stores post-RoPE K” as a universal definition. Cache representation is architecture- and implementation-dependent.*

## Why cache management becomes difficult

---

A single request has a simple story: its cache grows as tokens are processed. A serving system has many requests with different prompt lengths, generation lengths, arrival times, and completion times.

At any moment, the server may need to manage:

- one sequence that has just started;
- another already holding tens of thousands of cached tokens;
- several requests that finish and free memory;
- requests that share prefixes;
- requests that need different attention-window patterns.

This turns KV cache from a tensor into a dynamic memory-management problem.

The PagedAttention paper identified fragmentation and redundant duplication as major sources of wasted KV-cache capacity in contemporary serving systems, motivating a block-based design inspired by virtual memory.

# Why contiguous allocation wastes capacity

---

If every request needs one large contiguous KV allocation sized for its maximum possible sequence length, two kinds of waste can appear:

## Reserved-but-unused space

A request may reserve room it never reaches.

## External fragmentation

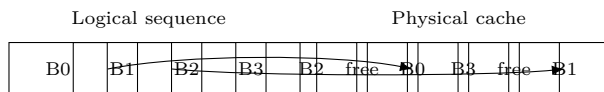
Free memory can exist in pieces that are individually too small to satisfy a large contiguous request even when their total size is sufficient.

The PagedAttention design avoids requiring each logical sequence's KV state to live in one contiguous physical region. Instead it divides cache state into fixed-size blocks and maps logical blocks to physical blocks.

*Paging is about memory management. It should not be confused with FlashAttention, which is primarily an IO-aware algorithm/kernel strategy for computing attention.*

## Logical sequence, non-contiguous physical blocks

PagedAttention applies an operating-systems-inspired idea to KV state. A sequence is divided into logical token blocks. Those logical blocks can reside in arbitrary available physical KV-cache blocks.



A block table tells the attention implementation where each logical block lives. Blocks can be allocated as the request grows and freed when no longer needed.

The original vLLM work also used block sharing to reduce duplication for related sequences.

## Paging has its own trade-offs

---

A KV block represents a fixed span of token positions. Smaller blocks can reduce internal waste at the tail of a sequence and make reuse more granular. Larger blocks can reduce bookkeeping and may better fit some kernel/memory-transfer choices.

There is no universal optimal block size independent of engine, hardware, workload, model, and kernel design.

Modern serving systems expose block-oriented cache managers, but their exact defaults and physical packing should be treated as implementation details rather than part of the definition of KV caching.

### The durable idea

logical token history  $\neq$  one contiguous physical allocation

That separation lets the runtime allocate, share, evict, and move cache state at block granularity.

## Reuse prefill state across requests

---

Ordinary KV caching reuses earlier state *within* an ongoing request. Prefix caching goes further: if a later request has a reusable prefix that matches previously computed state, the server can reuse cached KV blocks for that prefix instead of recomputing its prefill.

Current vLLM implements automatic prefix caching with block hashes based on the prefix and block content, plus additional inputs when necessary for correctness (for example, model/runtime-specific context).

### Two workloads that benefit

- repeated questions over the same long document or system prompt;
- multi-turn conversations where a large history is repeatedly sent as a prefix.

*Prefix caching is exact-state reuse for a matching computational prefix. It is not semantic similarity search and it is not RAG.*

## It reduces prefill work, not future decode work

---

Prefix caching can skip computation for a previously processed shared prefix. Once generation moves beyond that prefix, the new tokens still have to be decoded normally.

Current vLLM documentation states this explicitly: automatic prefix caching reduces query-processing/prefill time, but does not reduce the time required to generate new tokens in the decode phase.

That means a workload dominated by a very long generated answer may see little benefit even if its prompt prefix is reusable.

### Correctness requires more than matching text

A safe cache key may need to distinguish more than visible token text: model identity and configuration, adapters, multimodal inputs, or other execution state can affect whether a cached block is reusable. Exact requirements are runtime-specific.



## Reuse can reveal information through timing

---

In a multi-tenant service, cache hits can change latency. If unrelated users are allowed to share prefix-cache state, an attacker may be able to infer whether some content was already cached by observing timing differences.

Current vLLM provides optional per-request `cache_salt`. The salt is incorporated into the first block's cache identity so only requests that share the salt can reuse those blocks, creating a trust boundary for cache sharing.

The same engineering principle is broader than any one framework:

cache reuse domain    should match    trust domain.

Hash selection also matters: current vLLM documentation warns that non-cryptographic hashing increases collision risk and can create correctness or privacy concerns in shared environments.

## What happens when GPU cache is full?

---

A production cache has finite capacity. When new K/V blocks are needed, old reusable blocks may have to leave GPU memory.

There is no single universal eviction policy. Examples include:

- LRU-style eviction for reusable prefix blocks;
- priority-aware LRU variants;
- policies that protect blocks belonging to actively decoding requests;
- application-defined policies in extensible offloading systems.

Current vLLM prefix-cache design documents an LRU-based free/eviction path. Current TensorRT-LLM documentation describes prioritized LRU for reusable blocks.

*Evicting a reusable prefix block does not change model correctness. It changes whether future requests can avoid recomputing that state. Evicting cache required by an active sequence is a different matter and must be coordinated by the runtime.*

## Use slower memory as a larger cache tier

---

When GPU memory is scarce, reusable KV blocks can be moved to larger but slower storage and promoted back when needed.

A common hierarchy is

GPU HBM  $\leftrightarrow$  CPU memory  $\leftrightarrow$  secondary storage/network tier.

Current vLLM supports CPU KV offloading and a multi-tier offloading design in which secondary tiers are staged through CPU memory. Current TensorRT-LLM can offload reusable blocks to host memory and exposes connectors for external KV-cache storage or transfer.

### The trade-off

Offloading expands effective cache capacity, but introduces data movement. Whether it helps depends on reuse probability, transfer bandwidth/latency, overlap with compute, and the cost of recomputing the state instead.

*“Offload to CPU to save GPU memory” is incomplete. The real decision is: transfer cached state or recompute it?*

## Store K/V with fewer bits

---

Another way to reduce cache capacity and bandwidth pressure is to store K/V activations at lower precision.

Conceptually, if the tensor payload moved from 16-bit storage to ideal 8-bit storage, the element-storage component would halve. Real implementations also require scales, alignment, metadata, and backend-specific layouts, so practical memory savings need not equal the ideal bit-width ratio exactly.

Current vLLM supports multiple KV-cache storage dtypes, including FP8 variants on supported hardware/backends. It also documents calibration pathways for quantization scales.

Research systems such as KVQuant study even lower-bit KV representations, but their reported accuracy/throughput results are method-, model-, hardware-, and benchmark-specific.

## Compression changes numerical state

---

Unlike PagedAttention's memory placement or exact prefix reuse, KV-cache quantization changes the numerical representation of cached tensors.

That introduces quantization error into future attention computations. The engineering question becomes:

memory/bandwidth saved vs quality and kernel cost.

The right precision can depend on:

- model architecture and layer sensitivity;
- key/value activation distributions;
- context length;
- quantization granularity and scales;
- hardware support and available kernels.

Current vLLM even allows selected layers or attention types to remain unquantized while others use a lower-precision KV dtype, reflecting the fact that sensitivity need not be uniform.

## Shortcuts that create the wrong mental model

---

**“KV cache is the model’s memory.”**

No. It is temporary inference state derived from a token sequence at attention layers.

**“KV caching makes decoding constant-time.”**

No. It avoids recomputing historical K/V states; dense attention for the new query still interacts with an expanding history.

**“MQA/GQA reduce query heads.”**

No. Their defining change is the number/sharing of K/V heads relative to query heads.

**“PagedAttention is FlashAttention.”**

No. PagedAttention addresses KV-cache memory management; FlashAttention addresses efficient attention computation/memory traffic.

**“Prefix caching speeds every generated token.”**

No. It can skip reusable prefix prefill; it does not remove normal decode work for new tokens.

**“KV quantization is lossless compression.”**

Generally no. Lower-precision representation introduces numerical approximation.

## Five questions to ask about any KV-cache design

---

1. **What is stored?**

Which K/V or latent states, at which layers, in what dtype/layout?

2. **How much state is created per token?**

Track  $L, H_{KV}, D_h$ , bytes, attention-window rules, and sharding.

3. **Where does the state live?**

Contiguous GPU tensors, paged blocks, CPU offload, remote cache, or multiple tiers?

4. **When can it be reused?**

Within a request, across matching prefixes, across instances, or not at all?

5. **What is the bottleneck?**

GPU capacity, memory bandwidth, transfer latency, recomputation, fragmentation, or scheduler pressure?

These questions separate the mathematical KV-cache mechanism from the serving system built around it.

## KV cache in one chain

---

A causal Transformer repeatedly generates a new token from prior tokens. Past attention-layer states do not change when future tokens are appended, so the model can retain their keys and values instead of recomputing them.

For a conventional cache, tensor payload grows roughly as

$$2LTH_{KV}D_hb.$$

That makes context length and concurrent requests expensive. MQA/GQA reduce the number of KV heads; MLA compresses the cached representation; lower-precision storage reduces bytes per element.

At serving scale, dynamic sequence lengths create allocation and reuse problems. Block-based cache management separates logical token history from physical memory placement. Prefix caching reuses already-computed prefill state across compatible requests. Eviction and offloading decide which reusable blocks remain near the GPU. Quantization trades numerical precision for capacity/bandwidth savings.

*The KV cache is simultaneously a consequence of causal attention, a model-architecture design constraint, and a runtime memory-management problem. Understanding all three levels is what makes the concept useful.*



# Primary papers and current implementation sources

---

- [1] A. Vaswani et al. *Attention Is All You Need*. NeurIPS 2017. arXiv:1706.03762.
  - [2] N. Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. arXiv:1911.02150, 2019.
  - [3] J. Ainslie et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP 2023. arXiv:2305.13245.
  - [4] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. arXiv:2405.04434, 2024.
  - [5] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP 2023. DOI:10.1145/3600006.3613165; arXiv:2309.06180.
  - [6] NVIDIA Transformer Engine documentation. PyTorch attention/inference parameters; K/V caching during decoding. Consulted September 2026.
  - [7] vLLM documentation. Automatic Prefix Caching, cache configuration, and prefix-cache isolation/security. Current documentation consulted September 2026.
  - [8] NVIDIA TensorRT-LLM documentation. KV Cache System, reuse, prioritized eviction, and host offloading. Current documentation consulted September 2026.
  - [9] vLLM documentation. KV Offloading Usage Guide and multi-tier offloading APIs. Current documentation consulted September 2026.
  - [10] vLLM documentation. Quantized KV Cache and CacheConfig storage dtype options. Current documentation consulted September 2026.
  - [11] C. Hooper et al. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization*. arXiv:2401.18079, 2024.
-